

作业：Docker 容器化部署

作业目标

使用 Docker 完成项目的容器化部署，实现开发与生产环境一致，并通过 GitHub Actions 自动化部署流程。

第一步：编写 Dockerfile（必做）

为前端和后端分别编写多阶段构建的 Dockerfile。

推荐 Prompt（复制使用）：

为【你的技术栈，如 Node.js/Python FastAPI/Spring Boot】后端创建生产级 Dockerfile:

- 使用多阶段构建减小镜像体积（目标 < 200MB）
- 基础镜像使用 Alpine 或 slim 变体
- 非 root 用户运行
- 健康检查端点 /health
- 暴露端口 【你的端口】
- 包含 .dockerignore 文件

项目结构:

【粘贴你的项目目录结构】

后端示例（Python FastAPI，多阶段构建）：

```
FROM python:3.12-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /usr/local/lib/python3.12/site-packages \
    /usr/local/lib/python3.12/site-packages
COPY . .
# 非 root 用户
RUN useradd -m appuser && chown -R appuser /app
USER appuser
EXPOSE 8080
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

要求：

- ☐ 前端 Dockerfile（多阶段构建）
 - ☐ 后端 Dockerfile（多阶段构建）
 - ☐ `.dockerignore` 文件（排除 `node_modules`、`.git`、`.env` 等）
 - ☐ 非 root 用户运行
-

第二步：Docker Compose 配置（必做）

开发环境（`compose.yaml`）

创建支持热重载的开发配置：

```
# compose.yaml（无需 version 字段，使用 Docker Compose V2）
services:
  frontend:
    build: ./frontend
    ports:
      - "5173:5173"
    volumes:
      - ./frontend:/app # 热重载
    environment:
      - VITE_API_URL=http://localhost:8080

  backend:
    build: ./backend
    ports:
      - "8080:8080"
    environment:
      - DATABASE_URL=postgres://postgres:dev123@db:5432/myapp
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:17-alpine
    environment:
      POSTGRES_PASSWORD: dev123
      POSTGRES_DB: myapp
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
    volumes:
```

```
    - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

生产环境 (compose.prod.yaml)

创建包含资源限制和密钥管理的生产配置：

```
# compose.prod.yaml
services:
  app:
    image: ghcr.io/${GITHUB_REPOSITORY}:${VERSION:-latest}
    restart: unless-stopped
    ports:
      - "80:8080"
    environment:
      - DATABASE_URL=postgres://db:5432/prod
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "wget", "-q0-", "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 3
    deploy:
      resources:
        limits:
          memory: 512M

  db:
    image: postgres:17-alpine
    restart: unless-stopped
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    volumes:
      - pgdata:/var/lib/postgresql/data
    secrets:
      - db_password

volumes:
  pgdata:
```

```
secrets:
  db_password:
    file: ./secrets/db_password.txt
```

第三步：GitHub Actions 自动化部署（必做，选一项）

选项 A：构建并推送镜像到 GHCR（推荐）

```
# .github/workflows/docker.yml
name: Build and Push Docker Image

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write

    steps:
      - uses: actions/checkout@v4

      - name: Login to GHCR
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${github.actor}
          password: ${secrets.GITHUB_TOKEN}

      - name: Build and push
        uses: docker/build-push-action@v6
        with:
          context: ./backend
          push: true
          tags: ghcr.io/${github.repository}/backend:latest
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

```
- name: Scan for vulnerabilities
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: ghcr.io/${{ github.repository }}/backend:latest
    severity: CRITICAL,HIGH
```

选项 B：本地部署脚本（适合无服务器的情况）

创建 `deploy.sh`，证明可以一键启动：

```
#!/bin/bash
set -e

echo "🚀 开始部署..."

# 拉取最新镜像（如果使用 GHCR）
# docker compose -f compose.prod.yaml pull

# 重新构建并启动
docker compose -f compose.prod.yaml up -d --build

# 等待健康检查
echo "⌚ 等待服务就绪..."
sleep 10

# 显示服务状态
docker compose -f compose.prod.yaml ps
echo "✅ 部署完成"
```

部署检查清单

根据项目情况，完成以下检查：

基础配置

- ☐ `docker compose up -d` 可以正常启动所有服务
- ☐ 应用可以通过浏览器正常访问
- ☐ 数据库数据持久化（重启容器后数据不丢失）
- ☐ 健康检查配置正确（`docker compose ps` 显示 healthy）

安全配置

- ☐ 非 root 用户运行容器
- ☐ `.env` 已加入 `.gitignore`，仓库中有 `.env.example`

- ☐ 生产密码不硬编码在配置文件中
- ☐ 镜像安全扫描通过 (trivy 无高危漏洞, 或已记录已知漏洞原因)

镜像优化

- ☐ 使用多阶段构建
 - ☐ `.dockerignore` 排除无关文件
 - ☐ 镜像大小合理 (参考: 前端 < 50MB, 后端 < 300MB)
-

提交内容

Git 仓库提交

1. 前端 `Dockerfile + .dockerignore`
2. 后端 `Dockerfile + .dockerignore`
3. `compose.yaml` (开发环境)
4. `compose.prod.yaml` (生产环境)
5. `deploy.sh` 或 `.github/workflows/docker.yml`
6. `.env.example` (密钥配置模板)
7. `docs/contributions/11-docker/你的名字.md`

学习通提交 (截图)

1. Git 提交记录: `git log --author="YOUR_NAME" --oneline --graph` 终端截图
2. 服务运行截图: `docker compose ps` 显示所有服务 healthy
3. 应用访问截图: 浏览器访问部署的应用

💡 `docker compose up -d` 必须能正常启动, 这是基本要求

个人贡献说明

每位成员在 `docs/contributions/11-docker/` 下提交:

文件名: 你的名字.md

内容模板:

```
# Docker 部署贡献说明
```

```
姓名: XXX
```

```
学号: XXX
```

```
日期: XXXX-XX-XX
```

```
## 我完成的工作
```

1. Dockerfile 编写

- [] 前端 Dockerfile (多阶段构建)
- [] 后端 Dockerfile (多阶段构建)
- [] .dockerignore 文件

2. Compose 配置

- [] 开发环境 compose.yaml
- [] 生产环境 compose.prod.yaml
- [] 健康检查配置

3. 自动化部署

- 选择了选项 A / B:
- 具体内容:

PR 链接

- PR #X: <https://github.com/xxx/xxx/pull/X>

遇到的问题和解决

1. 问题: XXX
解决: XXX

AI 使用情况

- 使用了哪些 Prompt:
- AI 帮助解决了哪些问题:

心得体会

(简要说明在 Docker 部署过程中的收获)

评分标准

项目	分值	说明
Dockerfile 编写	25%	多阶段构建, 镜像体积合理
Compose 配置	25%	健康检查、依赖管理、持久化
服务正常运行	20%	<code>docker compose up</code> 可一键启动
安全配置	15%	非 root、密钥管理

项目	分值	说明
个人贡献说明	15%	贡献清晰，有截图证明

额外加分项

加分项	说明
GitHub Actions 自动构建	推送到 GHCR, Actions 绿色通过
trivy 漏洞扫描	CI 中集成, 无高危漏洞
镜像 < 150MB	充分优化 Dockerfile
日志集成	统一日志格式 / 使用 Loki 收集